Development Plan
Squigglr

Team #17, Squigglr
Rayan Nasrallah
Ahmed Elzaria
Muhammad Huzaifah
Shamil Canbolat
Moustafa Moustafa

Table 1: Revision History

Date	Developer(s)	Change
2026-09-26	Muhammad Huzaifah	Added Workflow Plan (Git, Issues, CI/CD), Coding Standard, Scheduling, Meeting Plan, Communication Plan, and License sections
2026-09-26	Ahmed Elzaria	Added team member roles, shared responsibilities, expected technology, and the team charter.
2026-09-26	Ahmed Elzaria	Added individual reflection responses and removed a duplicate reflection heading.
2026-09-26	Moustafa Moustafa	Added my reflection responses to the appendix.
2026-09-27	Ahmed Elzaria	Integrated the merged team sections and revised the response to reflection question three into a team response.
2026-09-27	Muhammad Huzaifah	Added my reflection responses to appendix.
2026-09-27	Ahmed Elzaria	Revised the proof-of-concept plan, AI-use guidance and disclosure, and team reflection. Clarified workflow, roles, licensing, and terminology; corrected cross-references and improved table placement and page layout following the checklist review.
2026-09-28	Moustafa Moustafa	Disclosed the use of AI to diagnose and fix missing references in the automatically built PDFs.
2026-11-01	Muhammad Huzaifah	Addressed peer review: named the candidate model kinds for creative enhancement and referred to the SRS comparison (#74).
2026-11-01	Shamil Canbolat	Changed the fall custom document from literature review report to specialist interview report.
2026-11-02	Moustafa Moustafa	Addressed peer review: stated that only two POC experiments concern creative drawing, and that the others do not depend on refinement (#70).
2026-11-11	Ahmed Elzaria	Addressed TA feedback: the lists in the Team Charter and the meeting procedure in the Team Meeting Plan are now bulleted or numbered lists, with no rule changed (#320).
2026-11-11	Ahmed Elzaria	Addressed TA feedback: the POC is now a hosted first version built around the square lesson, with two refinement experiments run on their own. The model experiment is narrowed to one model with a pass criterion set before the run, adaptive guidance is no longer an experiment, and a new subsection says what each failed experiment would mean (#319).

This Development Plan describes the team's approach to intellectual property, collaboration, task management, scheduling, and quality assurance. It outlines the proof-of-concept investigation, expected technologies, coding standards, and planned use of Artificial Intelligence (AI). The appendices disclose AI assistance used in preparing this document, present team reflections, and establish the Team Charter. Table 1 records the document's revisions.

1 Confidential Information?

The project does not currently involve confidential information supplied by an industry partner. Credentials and any private user data collected during development or evaluation will be kept out of the public repository and published documentation.

2 IP to Protect

The team's original code and documentation constitute Intellectual Property (IP). The project has no separate IP ownership or assignment agreement. The team's original work is licensed under the MIT License, as described in Section 3.

Third-party code, assets, and other materials remain subject to their respective licences and attribution requirements.

3 Copyright License

Squigglr adopts the MIT License for the team's own work. The complete terms and the copyright notice, "Copyright (c) 2026 Squigglr contributors," are in the repository's LICENSE file. MIT permits reuse, modification, distribution, and commercial use, provided the copyright and permission notices accompany copies or substantial portions of the work. The work is supplied without warranty. Contributors must preserve applicable third-party notices when reusing material; the team's license does not replace the terms of dependencies or other third-party content.

4 Team Meeting Plan

The weekly meetings in Table 2 provide planning, progress checks, and review time. All times use **America/Toronto**, so the local meeting time remains fixed through daylight-saving changes. Virtual meetings use Google Meet; Microsoft Teams holds invitations and reservations. The invitation specifies the format and duration, adjusted to the current deliverables. Physical locations are not published in this plan. Agreed changes or cancellations are recorded in the meeting issue and the invitation, with prompt notification through WhatsApp.

Each meeting follows these steps:

Table 2: Recurring meetings

Meeting	Recurrence	Primary purpose
Team planning	Tuesday, 5:00 p.m.	Plan upcoming work, allocate tasks, and review dependencies.
Team status	Saturday, 12:00 p.m.	Review each member's progress and blockers.
Team review	Sunday, 10:00 a.m.	Finalize and review the week's work.
Supervisor	Wednesday, 12:30 p.m.	Discuss project updates, deliverable questions, concerns, and feedback.

1. The **Project Manager** chairs meetings and posts an agenda in the relevant GitHub meeting issue at least 24 hours beforehand.
2. Members review the agenda and update their assigned issues before attending.
3. The meeting begins with progress and blockers, proceeds through the agenda and decisions, and ends with the chair confirming action owners and deadlines.
4. The designated notetaker records attendance, decisions, notes, and action items in the meeting issue's description within 24 hours after the meeting. Each action has an owner and deadline; substantive work receives a linked task issue.
5. The chair checks that these records capture the agreed actions before the meeting issue is closed.

If the chair or notetaker is unavailable, the team identifies a substitute in the meeting issue before the meeting. Members follow the absence and emergency policy in the Team Charter:

- notify the team of predictable absences at least 24 hours ahead where possible, and of emergencies as soon as reasonably possible;
- an absent member reviews the notes and confirms assigned actions within 24 hours of the notes becoming available, unless an emergency prevents this; the team then agrees on a recovery time.

5 Team Communication Plan

Table 3 assigns each communication tool a clear purpose. GitHub is the canonical record: whoever coordinates a consequential conversation records its decision, rationale, and resulting tasks in the relevant issue within 24 hours. Repository

Table 3: Communication tools and responsibilities

Tool	Purpose
GitHub	Canonical task records, technical decisions, meeting notes, code and document history, and reviews.
WhatsApp	General conversation, quick coordination, and urgent notifications linking to the relevant GitHub issue.
Microsoft Teams	File sharing and meeting invitations or reservations.
Google Meet	Live virtual meetings.
University email	Formal external correspondence through the Team Liaison. For the instructor and teaching assistants, put the course code in the subject and include the team number and name; contact the instructor by email.

deliverables remain version controlled; files shared in Teams supplement that record.

Members acknowledge routine actionable requests within 24 hours. Urgent blockers require acknowledgement within four hours counted during 9:00 a.m.–9:00 p.m. Toronto local time. For example, an urgent request at 8:00 p.m. is due for acknowledgement by noon the next day. An acknowledgement provides an answer or an expected response or action time; it does not promise resolution within that window. The sender raises an overdue urgent request directly with the Project Manager for coordination or reassignment. Communicated unavailability and emergencies follow the Team Charter.

6 Team Member Roles

All five team members are software developers and will contribute substantively to the product's code. Each member will also lead an area of the project, as shown in Table 4. Leads coordinate their areas, track outstanding work, and raise concerns with the team. Implementation tasks within each area will be shared according to workload, experience, and learning interests.

User Interface (UI) and User Experience (UX) responsibilities cover interaction design and usability. Development and Operations (DevOps) responsibilities cover development environments, build automation, and deployment.

Table 4: Team lead roles and responsibilities

Member	**Lead roles**	**Responsibilities**
Ahmed Elzaria	Project Manager and UI/UX Lead	Coordinate scope, task allocation, workload, and deadlines. Chair team meetings and follow up on blockers. Lead user flows, interaction design, and usability evaluation in collaboration with the Frontend Lead.
Rayan Nasrallah	Backend Lead	Coordinate backend design and implementation, including application logic, data storage, and interfaces with other components. Identify backend dependencies and support teammates working in this area.
Shamil Canbolat	Team Liaison and DevOps Lead	Coordinate communication with the teaching team and external stakeholders, and relay feedback and follow-up actions. Coordinate development setup, automated build and test workflows, and deployment.
Muhammad Huzaifah	Testing Lead	Coordinate the software testing strategy, integration testing, and defect follow-up. Identify gaps in test coverage and support teammates in designing and maintaining tests.
Moustafa Moustafa	Frontend Lead and Notetaker	Coordinate implementation of the user interface and its integration with backend functionality. Work with the UI/UX Lead to translate interaction designs into usable, consistent interfaces. Record meeting attendance, decisions, action items, and unresolved questions.

6.1 Shared Responsibilities

The following responsibilities apply to every member, regardless of their lead roles:

- **Software development:** Implement and maintain product features, resolve defects, and contribute to integration. Every member will receive substantive product-code tasks. Members may work across frontend, backend, and other technical areas with support from the relevant leads.
- **Testing:** Develop and run appropriate tests for their changes, investigate failures, and address defects. The Testing Lead coordinates the overall approach; each developer remains responsible for testing their own contributions.
- **Research and requirements:** Investigate stakeholder needs, domain knowledge, and technical approaches. Record supporting sources and assumptions, and contribute to defining and reviewing requirements.
- **Documentation:** Write and update the documentation associated with assigned work, including relevant requirements, design decisions, setup instructions, and test evidence.
- **Peer review:** Review teammates' code and documentation for correctness, clarity, and consistency with requirements and agreed standards. Each substantial change will be reviewed by someone other than its author before acceptance.
- **Task tracking and communication:** Keep assigned GitHub issues and project-board entries current, communicate progress and blockers, and notify the team when an agreed deadline is at risk.

Each substantive task will have a named owner, an agreed deadline, and completion criteria recorded in a GitHub issue. Tasks involving multiple components will also identify who is responsible for coordinating their integration. These assignments establish individual accountability within the team's shared responsibilities.

6.2 Meeting and Milestone Responsibilities

Ahmed will chair team meetings as Project Manager, with a substitute identified when he is unavailable. The chair will prepare the agenda, guide discussion, and confirm action owners and deadlines. Moustafa will serve as the team notetaker, recording attendance, decisions, action items, and unresolved questions in the meeting issue. When Moustafa is unavailable, another member will be assigned as the substitute notetaker before the meeting.

A milestone Quality Assurance (QA) coordinator will rotate each milestone among all five members. The coordinator will be named in the milestone's tracking issue and will ensure that its deliverables are reviewed against the

applicable course checklists and Avenue rubric. The coordinator will assign review actions, track corrections, and verify that outstanding findings are resolved before submission. Another member will review the coordinator's own contributions.

Milestone QA covers the completeness and consistency of the entire deliverable, including documentation and submission requirements. Huzaifah's ongoing Testing Lead role covers the software testing strategy and coordination of test activities. All members remain responsible for the quality of their own work under both arrangements.

6.3 Workload Balance and Role Changes

At the start of each milestone, the team will allocate tasks using estimated effort, member availability, and project priorities. Coordination, research, review, and liaison duties will be included when assessing workload. Members will support areas with heavier implementation demands, and the Project Manager will coordinate redistribution when tasks become blocked or workloads become uneven. Shamil and Huzaifah will receive product-feature implementation tasks alongside their DevOps, liaison, and testing responsibilities.

Lead assignments will be reviewed at each milestone and whenever availability, interests, or project needs change. Any member may request a reassignment during a team meeting or through a GitHub issue. The team will agree on changes using the decision-making process in the Team Charter and record the outcome in the meeting notes.

The outgoing lead will hand over current progress, outstanding actions, known issues, and relevant resources to the incoming lead. Table 4 and affected issue assignments will then be updated. If a lead becomes unexpectedly unavailable, the team will designate a temporary substitute and redistribute urgent work.

Members will share technical knowledge through repository documentation, peer review, and brief walkthroughs. This overlap will help members support one another and assume new responsibilities throughout the project.

7 Workflow Plan

GitHub connects each task to its implementation, validation, and review. The workflow below applies to code, documentation, and repository maintenance. Authors own their changes through review and closure; reviewers check both the completion criteria and the evidence supplied.

7.1 Git Branches, Commits, and Review

The planned workflow uses short-lived issue branches, `development` for integration, and `main` for stable versions. Until the integration branch and its checks are configured, contributions use issue branches based on `main` and pull requests targeting `main`.

The DevOps Lead will coordinate creation of `development`, extend the relevant checks to that branch, and update the repository instructions before the team adopts the integration workflow. The same review requirements apply during both stages: at least one teammate other than the author must approve the contribution, and applicable checks must pass. These are team requirements regardless of whether repository settings enforce them.

Once the integration branch and its checks are ready, the team will use the following workflow:

1. Select an assigned GitHub issue with clear completion criteria. Update local `development` from the remote, then create a branch using this pattern:

   ```
   <type>/<issue-number>-<short-description>
   ```

   For example, use `docs/14-workflow-process`. Move the issue to *In progress*.

2. Make focused commits that preserve each contributor's work. Use the commit format below and include the issue reference. Keep unrelated changes in separate issues and branches.

3. Run checks relevant to the change and inspect the results. Push the branch and open a pull request (PR) against `development`. Include the evidence needed to review it and move the issue to *In review*.

4. Address feedback, obtain at least one teammate's approval, and resolve review conversations. All applicable checks must pass. Substantive changes after approval require renewed review before merging.

5. Merge into `development` using **Create a merge commit**. Confirm the issue's completion criteria, update the board, and delete the merged issue branch. Close work completed at this stage manually with a link to its PR; work requiring stable integration remains open.

6. The affected contributors verify their combined changes on `development`, including applicable integration checks and manual user-flow or document review. Fix integration failures through reviewed issue branches before promoting the result.

7. When integration is verified, open a PR from `development` to `main` with the included issues and validation evidence. Obtain at least one approving teammate other than the author, pass all applicable checks, and resolve review conversations. Substantive changes require renewed review. Merge using **Create a merge commit**, verify the stable result, and close the remaining completed issues. Keep the `development` branch for the next integration cycle.

Allowed commit types are `feat`, `fix`, `docs`, `style`, `refactor`, `test`, `build`, `ci`, and `chore`. Use this format:

```
<type>: <short action describing the change> (#<issue-number>)

<optional explanation of why the change was needed>
```

The PR title summarizes the ticket and includes its issue number. Its description explains what changed and why, supplies validation evidence (checks run, document review, screenshots, or preview links as applicable), links the relevant issues, and identifies outstanding work. Use `Closes #<issue-number>` only when the entire issue is satisfied; otherwise use `Refs #<issue-number>`. GitHub's automatic issue closure applies when the closing PR merges into the default branch, `main`; close an issue manually if its completion criteria are fully satisfied on `development`. The approving teammate must be someone other than the author. Documentation reviewers use the Development Plan Checklist and Writing Checklist, as applicable, and inspect the generated Portable Document Format (PDF) file.

Merge commits preserve individual commits and attribution. Squash merging and history rewriting on shared branches are prohibited by team policy. To resolve integration conflicts, coordinate with affected contributors, merge the current target branch into the source branch, resolve conflicts, and rerun relevant checks. This means merging `development` into an issue branch, or `main` into `development` for a promotion PR. Correct mistakes already merged through a reviewed revert or corrective PR.

For each submitted version, the milestone Quality Assurance (QA) coordinator checks the generated PDF and identifies the exact submitted commit with an annotated tag, for example `submission/dev-plan-2026-09-28`. Record the tag and commit in the deliverable's tracking issue. Preserve existing tags; use a new revision suffix, such as `-r2`, for a corrected submission.

7.2 Issues and Project-Board States

Use the five existing templates as described in Table 5. When none fits, create a blank issue for development, documentation, maintenance, or a bug. Every substantive task records its purpose, relevant context, completion criteria, an accountable assignee, labels, dependencies (or explicitly none), and a target date. Add it to the project board and the applicable milestone. Bug reports also include reproduction steps and expected versus actual behaviour.

Apply labels from Table 6 to make work easy to find. When creating an issue, select exactly one priority label alongside the other applicable labels: `High Priority` for urgent tasks, `Medium Priority` for current deliverable tasks, or `Low Priority` for future deliverable tasks. Update that label when the task's urgency or planned deliverable changes. The `meeting` and `review` labels match the existing templates. Disposition labels accompany an explanation when closing an issue, including a link to the original issue when marking a duplicate.

The assignee maintains the board state using Table 7. A meeting issue reaches *Done* after the meeting occurs and its notes and actions are recorded.

Table 5: Existing issue templates

Template	**Use**
Lecture	Attendance, lecture topics, questions, and relevant notes.
Team Meeting	Team agenda, attendance, decisions, and action items.
Supervisor Meeting	Supervisor or stakeholder agenda, questions, feedback, and actions.
TA Meeting	Teaching Assistant (TA) deliverable feedback, questions, attendance, and follow-up work.
Peer Review	Feedback from another team about a specific artifact.

Table 6: Issue-label categories

Category	**Labels**
Work classification	`bug`, `fix`, `enhancement`, `documentation`, `repo-maintenance`, `accessibility`
Coordination and feedback	`meeting`, `review`, `question`
Assistance	`help wanted`, `good first issue`
Priority (choose one)	`High Priority`, `Medium Priority`, `Low Priority`
Disposition	`duplicate`, `invalid`, `wontfix`

Unfinished substantive actions remain in linked task issues, so closing a meeting record does not discard its follow-up work.

Table 7: Project-board state rules

Column	Entry or completion rule
Backlog	Identified work still needs clarification, prioritization, or scheduling.
Ready	Completion criteria are clear, dependencies are understood, and work is ready to be assigned or started.
In progress	An assignee is actively doing the work.
In review	Reviewable work and validation evidence are available.
Done	Completion criteria are satisfied and required review, merge, or submission is complete.

Priority labels are the issue's authoritative priority record. The project's existing Priority field is optional board metadata, separate from issue creation. When used, manually keep P0 aligned with `High Priority`, P1 with `Medium Priority`, and P2 with `Low Priority`; no automatic synchronization is configured. Size and Estimate are also optional. Record blockers and their dependencies in the issue and raise them promptly with the Project Manager; an issue remains in its current state with the blocker explained until it can proceed.

7.3 Continuous Integration and Delivery

Continuous Integration (CI) checks proposed changes; Continuous Delivery (CD) prepares a reviewed candidate for an authorized release. Existing automation and planned application checks are distinguished below.

Existing Documentation Automation

The repository's `latex-pages.yml` builds affected documentation on relevant PRs targeting `main` and pushes to `main`. Its path filters cover documentation, references, the Makefile, and documentation automation. On pushes to `main`, it updates committed PDFs and the dependency manifest. It also publishes the PDFs through GitHub Pages on non-PR runs, including manual dispatch. Compilation checks syntax; authors and reviewers must still inspect content, links, and layout. Contributors compile PDFs locally for review but do not commit them; the workflow updates `pdfs/` after merging to `main`. Until then, review the local build or the PR's GitHub Actions artifact, rather than expecting the committed PDF to contain the proposed source changes. Before accepting a documentation check, confirm that its log lists the changed document as compiled and that its PDF artifact matches the reviewed source. A successful status alone is insufficient if the run reused a cached PDF without rebuilding the changed document.

Automatically generated PDF and dependency-manifest updates are an exception to the contributor pull-request process. The documentation workflow publishes these outputs after the source changes have been reviewed and merged. This exception applies only to generated outputs; changes to document source or workflow configuration still require a reviewed pull request.

Planned Application Checks

When the application scaffold exists, the DevOps Lead will introduce the small pipeline in Table 8, with the Testing Lead coordinating test selection. These checks are planned; the application tooling is not yet installed. Authors run applicable checks before review; GitHub Actions will run them on PRs to `development` and `main`, and after merges to either branch. Committed package scripts and the dependency lockfile will make commands reproducible. Validation jobs use read-only permissions; deployment permissions are granted only to jobs that need them.

Table 8: Planned application validation pipeline

Check	**Tool and purpose**
Formatting	Prettier check mode detects formatting differences.
Lint	ESLint checks Next.js and TypeScript rules.
Type checking	TypeScript strict checking detects type errors.
Unit/component tests	Vitest and React Testing Library check logic and component behaviour.
Coverage report	`@vitest/coverage-v8` highlights untested code; reviewers interpret coverage alongside test quality.
Build	A Next.js production build checks application integration.
Browser smoke tests	Playwright checks a complete user flow once one exists; expand checks as flows become available.

Documentation compilation will remain separate from application checks. The DevOps Lead will ensure that required checks report an appropriate result on every applicable pull request, including when no relevant files have changed. GitHub's workflow syntax reference explains that skipping an entire workflow through path filters can leave required checks pending.

Before the team begins targeting `development`, documentation validation will be extended to that branch. Generated PDFs will continue to be committed and published from `main`. Any enforced pull-request protection must accommodate this publication process before it is enabled.

Change detection must report an error when its comparison commit is unavailable rather than treating the failure as an empty change list. These workflow improvements remain planned until implemented and verified.

Preview and Production Release Procedure

The planned Vercel setup provides automatic previews and manual production promotion. Reviewers inspect the preview; a release candidate must build successfully from reviewed `main` after the integration PR from `development` is approved and merged. The milestone QA coordinator verifies applicable automated checks, unresolved defects, and manual checks on target touchscreen and stylus hardware before relevant releases. The Project Manager authorizes promotion, and the DevOps Lead records the deployed commit, verification evidence, and release decision in GitHub. After promotion, the QA coordinator checks the deployed user flow. If verification fails, the DevOps Lead restores the previous working deployment and opens a corrective issue linked to the failed release.

8 Project Decomposition and Scheduling

The Squigglr repository holds project artifacts and their history. The Squigglr Project Kanban Board connects issues, ownership, priorities, milestones, and dates to the workflow in Section 7.

8.1 Decomposition and Weekly Planning

Break each deliverable into manageable issues with an owner, completion criteria, dependencies, and a target date. Use parent and sub-issues where they clarify a larger deliverable; identify an integration owner when work spans contributors. Supporting tasks retain their own deadlines rather than inheriting only the final submission date.

Members update their issues before meetings. Tuesday planning selects work using dependencies, availability, and the Team Charter's approximate ten-hour contribution per member per week, including scheduled course activities. Account for research, coordination, implementation, and review when allocating work. The Project Manager coordinates reallocation when blockers or uneven workloads threaten the plan.

Use the board's existing views for distinct purposes:

- **Backlog**: track movement through the work states in Table 7.
- **Priority board**: triage work using the issue's priority label; if the optional P0/P1/P2 field is populated, keep it aligned manually.
- **Team items / My items**: inspect ownership and individual workload, including review responsibilities.
- **Roadmap**: inspect start and target dates once those fields are populated.

Retain the existing milestone, Problem Statement, POC Plan, Development Plan. The Project Manager will create a milestone for each subsequent major deliverable, including separate milestones for distinct deliverables sharing a date,

such as the Software Requirements Specification (SRS) and Hazard Analysis. Future milestones have yet to be created. Link supporting tasks, reviews, and integration work to their applicable milestone.

Aim for review-ready contributions at least **48 hours before each course deadline**, leaving time for integration and corrections. The team must record agreed exceptions before the existing internal deadline passes. The milestone QA coordinator tracks checklist review and unresolved findings; another teammate reviews the coordinator's own work before submission.

8.2 Course Timeline and Schedule Maintenance

Table 9 follows the course Google Calendar, checked September 26, 2026, using its teaching-week labels and recurrence exceptions. Reading breaks are excluded from week numbering. Dates use America/Toronto; all-day entries do not establish a submission time. The Project Manager checks calendar changes during weekly planning and updates affected issues, milestone plans, and this document. Confirm submission times from course instructions and record them in issues before calculating internal review deadlines.

Proof of Concept (POC) and Verification and Validation (V&V) activities appear alongside document submissions and peer reviews. The proposed custom documents are the *Specialist Interview Report* for fall and the *Usability Report* for winter. These choices remain proposed until finalized. Demonstration windows are not individual appointments; the team's assigned slot will be recorded when available. Tentative calendar entries below remain subject to confirmation.

Table 9: Course deliverables and required activities, 2026–2027

Week	Date	Deliverable or required activity
2	Sept. 18, 2026	Team formation, project selection, and approved onboarding.
3	Sept. 22	Biosheets.
3	Sept. 23	Informal Problem Statement/Development Plan review.
3	Sept. 25	Peer-review assignment requests.
4	Sept. 28	Problem Statement and Goals, POC Plan, Development Plan, and Team Charter.
4	Oct. 1	Peer review of initial planning documents.
5	Oct. 7	Informal SRS and Hazard Analysis presentation.
6	Oct. 19	SRS and Hazard Analysis.
6	Oct. 22	SRS peer-review issues.

Continued on next page

Table 9: Course timeline (continued)

Week	Date	Deliverable or required activity
7	Oct. 28	Informal V&V and POC planning presentation.
8	Nov. 2	V&V Plan.
8	Nov. 5	V&V peer review and POC scheduling requests.
9	Nov. 11	Informal Design Revision −1 / POC discussion.
10	Nov. 16	Design Revision −1 and fall custom document (proposed Specialist Interview Report).
10	Nov. 19	Design peer review; POC contribution survey and productivity report.
11–12	Nov. 23–Dec. 4	POC demonstration window; individual slot assigned later.
13	Dec. 9	Code-review interviews.
15	Jan. 12, 2027	Informal design-document review.
16	Jan. 18	Design documentation; Rev 0 scheduling requests.
16	Jan. 21	Peer module implementation.
17	Jan. 25	Peer module reflection.
17	Jan. 27	EXPO registration (tentative).
17	Jan. 29	Rev 0 contribution survey and productivity report.
18–19	Feb. 1–11	Scheduled Rev 0 presentation sessions.
20	Feb. 23	Code-review interviews.
21	Mar. 1	Final-demo scheduling requests.
21	Mar. 2	Informal V&V Report and winter custom-document review.
22	Mar. 8	V&V Report and winter custom document (proposed Usability Report).
22	Mar. 11	V&V/winter custom-document peer review.
23	Mar. 15–21	Final demonstration sessions.
23	Mar. 18	Rev 1 contribution survey and productivity report.
25	Mar. 31	Final documentation; poster and video deadlines remain tentative.
26	Apr. 6	EXPO (tentative date/time).

Continued on next page

Table 9: Course timeline (continued)

Week	Date	Deliverable or required activity
26	Apr. 7	Course evaluation (tentative).

9 Proof of Concept Demonstration Plan

The Proof of Concept (POC) will investigate the implementation risks that could prevent Squigglr from meeting its core goals. It has two parts:

- A first working version of Squigglr, built around one guided lesson, the square, and hosted so that the specialists can try it on their own time and give feedback. The project supervisor suggested this form, and the teaching team agreed that a larger POC is justified when it is useful to the project. The app is then extended at each milestone.
- Two small refinement experiments, run on their own rather than in the hosted app.

The app is not a small version of the whole product. It holds only what the square lesson needs in PH-1 (SRS Section 21): no progress summary, no adult accounts and no creative space. The line and circle lessons, which the SRS also plans for PH-1, follow the same rules and are stored as data, so they are added once the square lesson works.

9.1 Main Implementation Risks

- **Editable drawing refinement:** Automatic refinement must produce drawing elements that children can continue editing. Generating a separate image would not meet this goal. The main uncertainty is whether supported drawings can be refined into valid paths while retaining the original strokes and allowing the refinement to be undone.
- **Model-based creative enhancement:** A pretrained model may return invalid drawing instructions, misinterpret the input, or substantially alter the intended drawing. Geometric shape correction alone would not establish that this model-based approach is feasible.
- **Drawing evaluation:** The fixed criteria must tell a complete square that meets `STAGE_PASS` from an incomplete or inaccurate one, at every drawing stage, and the attempt records must keep what happened. If they cannot, no lesson can advance a child correctly.
- **Touchscreen and stylus interaction:** Input coordinates, stroke continuity, and drawing responsiveness may vary across devices and browsers. Using an established drawing library reduces some implementation work

but does not establish that its integration will behave correctly on the target hardware.

- **Fit with practice:** The specialists may find that the lesson flow does not suit the children they work with. Finding this out from a working lesson before Rev 0 costs far less than finding it out from the finished product.

The rules for moving between stages, extensions and go-back suggestions are not a feasibility risk. They are fixed logic, and the V&V Plan tests them (T-LRN-016, T-LRN-017 and T-LRN-020).

Editable refinement is the main technical uncertainty. The initial investigation will use a small, agreed set of two-dimensional shapes and simple objects. Isometric drawing and unrestricted enhancement of arbitrary drawings are outside the POC scope.

Only the refinement experiments (2 and 3 below) concern creative drawing. The input experiment and the square lesson (1 and 4) cover what the guided lessons and records rely on, and do not depend on refinement, so a refinement risk that stays unresolved does not hold back the other core goals.

9.2 Demonstration and Evidence

The demonstration will present the following experiments and their results. The team will record the sample inputs, expected behaviour, observed results, and remaining limitations.

1. **Capture and edit drawing input.** Draw using touch and a compatible stylus on the target tablet. Repeat the relevant interactions on an additional supported device and browser combination. Record the hardware and browser versions used. Check that strokes follow the input position, remain continuous, and can be edited after capture. Observe responsiveness and record any delays or input failures. Opening the page without testing drawing interactions will not count as evidence of input compatibility.

2. **Apply reversible geometric refinement.** Draw examples from the agreed supported set and apply a geometric correction. Show that the result consists of editable paths on the same canvas. Modify the refined result, then undo the refinement and compare the restored drawing with the saved original. This experiment will assess path manipulation and recovery independently of a model.

3. **Investigate model-based enhancement.** Evaluate one candidate pretrained model on the agreed supported drawings, with a limited set of structured drawing instructions. Before the run, the team records in the POC issue the share of responses that must be valid operations the application can apply, for the experiment to pass. Demonstrate that an invalid instruction is rejected and leaves the drawing unchanged. Record successful and unsuccessful examples, response times, and estimated request costs. Report this evidence separately from the geometric refinement results.

4. **Run the square lesson end to end.** On the hosted app and the target tablet, run the square lesson through its demonstration and its three drawing stages. Use prepared passing, failing, incomplete and interrupted attempts to show that each is evaluated as expected, that a pass reduces the guidance by moving to the next stage, and that the attempt records keep the strokes, the stage and the outcome. Then share the app with the specialists, and record their feedback for the demonstration.

Before running the experiments, the team will record the supported examples, the square lesson's required parts and endpoints, and the evaluation criteria. A risk will remain unresolved if the observed behaviour does not satisfy those criteria. The demonstration will include limitations and unsuccessful cases rather than relying only on selected successes.

Prepared attempts will test the correctness of the evaluation and the lesson rules. Neither they nor the specialists' first impressions will establish that the lessons improve children's visual-motor skills. That question requires later stakeholder evaluation and is not a prerequisite for this technical demonstration.

9.3 If an Experiment Fails

Each experiment tests a risk to a different goal, so a failure changes the project in a different way:

1. **Input.** Every goal depends on drawing input, so this is the most serious failure. The team would narrow the supported devices and browsers, or replace the drawing library, before Rev 0.

2. **Geometric refinement.** Editable refinement (G.4) could not be met as specified. The team would narrow the supported drawing set to the shapes where refinement works. The guided lessons are not affected.

3. **Model-based enhancement.** The model would not be used. Creative refinement would use geometric correction only (experiment 2), and further model investigation would be deferred. No guided practice goal depends on it.

4. **Square lesson.** If the criteria cannot tell a complete square from an incomplete one, the pass criteria are reworked before more lessons are added. If the specialists find the lesson flow unsuitable, the lesson design changes before Rev 0, which is why the app is shared with them.

Changes affecting core goals will be agreed with the supervisor and reflected consistently in the Problem Statement and Goals document and this plan.

9.4 Preparation and Follow-Up

The POC demonstration window is Nov. 23 to Dec. 4. Preparation will proceed through the following checkpoints:

- Confirm the target hardware, the square lesson's required parts and endpoints, the supported refinement examples, the evaluation criteria, and the pass criterion of experiment 3.
- Build and host the square lesson, and run the refinement experiments independently.
- Share the hosted app with the specialists, leaving them time to try it before the demonstration.
- Review the evidence and the specialists' feedback, identify unresolved risks, and agree on the demonstration content.
- Rehearse on the target hardware and retain the results and known limitations for discussion with the supervisor.

The Project Manager will coordinate task ownership and checkpoints. The relevant technical leads will coordinate implementation, the Testing Lead will coordinate evaluation, and all members will contribute through assigned development and review tasks.

After the demonstration, the hosted app becomes the starting point for Rev 0 and is extended at each milestone, following the phase plan in SRS Section 21.

10 Expected Technology

Following Dr. Anand's recommendation, Squigglr is planned as a web application supporting touchscreen and stylus drawing. The expected technologies favour a small, consistent stack that the team can develop within the capstone timeline. These initial choices may change following technical investigation and proof-of-concept results; they do not define the project's functional requirements.

10.1 Application Technologies

Table 10 summarizes the expected application technologies, their purposes, and their main trade-offs.

Table 10: Expected application technologies

Technology	**Planned use**	**Benefits and limitations**
TypeScript	Implement client-side and server-side application logic.	Shared types help maintain consistent data structures. Type checking does not replace runtime validation or testing.
React and Next.js	Build activity screens, progress views, and backend endpoints within one application.	Reusable components and a single project simplify development. Members must learn the client/server boundary.
Paper.js and Pointer Events	Represent and render editable drawing paths, and capture touchscreen or stylus input through browser events.	Path manipulation supports geometric enhancement. Integration and input behaviour require device testing.
Tailwind CSS	Use Cascading Style Sheets (CSS) utilities to style responsive screens and shared interface components.	Shared conventions support visual consistency. Members must learn the utility-class syntax.
Supabase	Use PostgreSQL for drawing and progress records, and Supabase Auth for adult account authentication.	Managed services reduce infrastructure work. Access policies require configuration and testing.
Vercel	Host the application and provide preview deployments for reviewing changes.	Next.js integration simplifies deployment. Service limits and potential costs must be monitored.

10.2 Drawing Assessment and Enhancement

The application is expected to retain drawings as editable paths and coordinates, with the canvas displaying those paths. Enhancement will update this drawing data and redraw the same canvas, allowing the user to continue working on the result. The original strokes will be preserved separately for assessment and to support undoing an enhancement.

Paper.js provides path smoothing and simplification that the team expects to reuse. The team will implement the application-specific logic for recording strokes, assessing drawings, adjusting activity difficulty, and managing enhancement actions. Initial assessment will consider explicit measures such as deviation from a target path and shape coverage. Initial difficulty adjustment will use rules based on activity performance.

Basic geometric enhancement will be investigated first because it can operate directly on the existing paths without model training. Smoothing and simplification will be described as geometric processing, rather than artificial intelligence.

For creative enhancement, the team will investigate an existing pretrained multimodal model capable of accepting a drawing preview and returning structured drawing instructions. Candidate instructions include modifying a curve or adding a supported shape. The application would validate these instructions before applying them to the editable drawing. A service that returns only a bitmap image would not directly satisfy this requirement.

No specific model has been selected, and custom model training is not currently planned. Model selection will depend on a focused evaluation of drawing preservation, output validity, response time, and cost. Creative vector enhancement remains an implementation uncertainty to investigate through the proof-of-concept work.

The candidate models are compared in the Reusable Components subsection of the Software Requirements Specification (SRS), which selects none of them. They fall into three kinds. Hosted multimodal models reached through an application programming interface (API), such as the OpenAI and Anthropic APIs, accept drawing images but send the drawing to an outside provider and charge per request. Open-weight vision-language models that the team could host itself, such as Qwen2.5-VL-7B-Instruct, keep drawings away from outside providers but need a machine with a graphics processing unit (GPU). Stroke-based generative models, such as Sketch-RNN, produce editable strokes but generate new drawings rather than adjusting the child's. The SRS also notes that the Gemini API's terms forbid its use in services likely to be accessed by people under 18.

10.3 Development and Verification Tools

Table 11 lists the expected tools for collaboration, code quality, and verification. Shared configurations and a dependency lockfile will help keep development environments consistent.

Table 11: Expected development and verification tools

Tool	**Planned use**	**Benefits and limitations**
Git, GitHub, and GitHub Projects	Version control, pull-request reviews, issue tracking, and milestone planning.	Connects tasks to reviewed changes. Requires consistent updates from all members.
ESLint and Prettier	Check code rules and maintain consistent formatting.	Automates routine checks. Does not establish functional correctness.
Vitest and React Testing Library	Test application logic and interface components. Use `@vitest/coverage-v8` to measure code coverage.	Supports focused automated tests. High coverage alone does not establish test quality.
Playwright	Test complete user flows across browser engines.	Checks interactions between components. Emulation cannot replace physical stylus testing.
GitHub Actions	Run automated checks on proposed changes.	Provides repeatable feedback before merging. Workflows require setup and maintenance.
Chrome DevTools	Investigate drawing responsiveness, rendering work, and slow interactions.	Helps locate performance bottlenecks. Results must also be checked on target hardware.
LaTeX	Produce course documentation from version-controlled source.	Supports consistent formatting and cross-references. Compiled documents still require visual review.

The Continuous Integration (CI) workflow is expected to run TypeScript checks, linting, formatting checks, unit tests, and a production build on pull requests. Automated browser tests will be added as complete user flows become available. Documentation changes will also undergo a LaTeX build check. Human review will complement these checks.

10.4 Early Learning and Feasibility Checks

Early technical work will focus on Paper.js path manipulation, touchscreen and stylus handling, Next.js client/server behaviour, Supabase access policies, and automated testing. A small experiment on a target tablet will check input accuracy, drawing responsiveness, and whether enhanced paths remain editable. Findings will inform the proof-of-concept plan and any changes to the expected technology.

11 Use of AI and LLMs

The team may use Artificial Intelligence (AI) tools, including Large Language Models (LLMs), to support research, documentation, and software development. Expected tools include ChatGPT and Claude for discussion and drafting, and GitHub Copilot or Kiro for coding assistance. Team members remain responsible for understanding, verifying, and maintaining any material they include in the project.

For documentation, the team will establish the project ideas, goals, and substantive decisions through its own discussion and analysis. AI tools may help articulate these ideas by turning team-provided notes and explanations into draft text, improving organization and clarity, and refining grammar and LaTeX formatting. Any additional suggestions from these tools will be treated as proposals for team evaluation rather than established project decisions.

Authors will verify that the resulting text accurately expresses the team's intended meaning and does not introduce unsupported claims or commitments. They will check it against course instructions, agreed decisions, and relevant source material. Suggested references must be located and checked for support. Personal reflections must represent each member's own experiences and conclusions, even when AI assists with their expression.

For software development, AI tools may suggest implementations, refactoring, tests, or explanations of unfamiliar code. The assigned developer will inspect and understand generated changes, check their compatibility with the existing design, and run the applicable checks described in Section 7.3. Tests will be checked against requirements and expected behaviour rather than assuming generated code and generated tests validate one another. Another team member will review the contribution before it is merged.

Team members will not include credentials, private user data, or confidential third-party material in prompts. Relevant usage terms and licensing conditions will be checked before incorporating external material or generated contributions.

Material AI assistance will be disclosed in the relevant deliverable according to course requirements. The disclosure will identify the tools used, the affected sections or artifacts, and how the resulting material was verified. The proposed use of a model within Squigglr's drawing enhancement feature is discussed separately in Section 10.

12 Coding Standard

Squigglr's Next.js/TypeScript standard uses the official Next.js Core Web Vitals and TypeScript ESLint configurations, with Prettier controlling formatting. This follows the Next.js ESLint configuration guidance and matches the planned application tools. Application enforcement is planned in Section 7.3; these tools are not yet installed.

12.1 Application Code and Formatting

Use TypeScript for application logic, with strict type checking enabled. Use the `.tsx` file extension for React components containing markup. Use `camelCase` for variables and functions and `PascalCase` for components and types. Choose descriptive names and small, focused functions. Comments explain non-obvious decisions, assumptions, or constraints rather than restating the code. Exceptional use of `any` or a lint suppression requires a documented justification next to the exception and explicit review.

The planned ESLint configuration combines these rule sets:

- `eslint-config-next/core-web-vitals`
- `eslint-config-next/typescript`

Use Prettier defaults for supported application and configuration files, and `eslint-config-prettier` to avoid conflicting formatting rules. Committed scripts will run ESLint, Prettier check mode, and TypeScript strict checking locally and in CI. Reviewers check naming, function scope, comments, and justified exceptions; automated checks cover only the rules enabled in the committed configuration.

12.2 SQL, Python, and Documentation

For authored Structured Query Language (SQL) migrations, use `snake_case` identifiers and uppercase SQL keywords. The Backend Lead and a teammate review schema and access-policy changes for their effect on stored data and permitted access. The existing Python automation helpers follow Python Enhancement Proposal (PEP) 8; reviewers check these conventions until dedicated Python linting is introduced.

For LaTeX, follow the course writing conventions and checklists. Before requesting review, authors compile changed documents and inspect their PDFs

for layout, working links, and unresolved references. The documentation workflow checks compilation; Python, SQL, and LaTeX conventions require manual review until dedicated automated checks are introduced.

13 Appendix — Generative AI Use Disclosure

This appendix describes how AI tools assisted in preparing and reviewing this document. The team remains responsible for its content and the decisions recorded in it.

13.1 AI Tools Used

Anthropic's Claude Opus 5.5 and OpenAI's GPT-6 Astra and GPT-5.6 Luna.

13.2 How and Where Used

Most team members first developed the ideas and concepts they wanted to express in their assigned sections, then used AI tools to help communicate them coherently. Members provided project context, notes, and explanations, and used the resulting drafts to improve organization, clarity, and flow. AI also assisted with grammar and LaTeX formatting.

The team used AI to research and compare potential technologies for Squigglr, considering the project goals, editable drawing requirements, available development time, and relevant industry practices. Suggested tools, trade-offs, and implementation approaches informed several planning and technology decisions. The team evaluated these suggestions when selecting the approaches documented in this plan.

AI also helped structure the proof-of-concept plan and review the combined document against the course checklists. This assistance included identifying inconsistencies, clarifying implementation risks and demonstration evidence, and suggesting corrections. The team retained responsibility for evaluating these suggestions and deciding which changes to adopt.

The team also used Claude Opus 5.5, through Claude Code, to diagnose a failure in the documentation workflow described in Section 7.3. Citations in the automatically built PDFs appeared as "[?]" and their reference lists were missing. AI traced the cause to the `bibtex` command being unavailable when the workflow restored LaTeX from its cache, and proposed a change to the workflow file (`latex-pages.yml`) that restores the command and stops the build if it is missing. The team reviewed the change and confirmed in a pull-request build that the references were restored.

13.3 Verification of AI-Generated Content

Authors review AI-assisted wording against their intended meaning, the team's agreed decisions, and the course instructions. Suggestions that introduce new commitments require team agreement; generated wording alone does not establish a project decision. Members check their own reflection responses to ensure that they accurately describe their experiences and conclusions.

Technical claims and suggested references are checked against relevant source material. Reviewers compare related sections for consistency and inspect the com-

piled PDF for layout problems and unresolved references. Successful compilation alone does not establish that the content is correct or complete.

Before submission, the team will resolve outstanding review findings and confirm that this disclosure accurately represents the assistance used. The team accepts responsibility for the accuracy, completeness, and appropriateness of the final document.

Appendix — Reflection

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. **Why is it important to create a development plan prior to starting the project?**

   Ahmed Elzaria:

   Creating the development plan helped me recognize that agreeing on a project idea does not automatically mean we agree on how to complete it. A team can share the same goals while having different expectations about responsibilities, deadlines, and acceptable contributions. Discussing these expectations early gives us a common basis for coordinating work and addressing problems before they affect a major deliverable.

   While developing the team roles, I initially considered assigning each person a narrow area of implementation. However, I realized that this could restrict members' learning and create dependence on individuals. Making every responsibility shared introduced the opposite problem: accountability became unclear. The approach I settled on combines named leads with shared development duties and explicit task ownership. This showed me that planning involves balancing competing needs, rather than simply dividing a list of tasks.

   The technology discussion also demonstrated the value of planning. Clarifying that enhanced drawings should remain editable changed what we needed from the implementation: returning a generated image would not satisfy that expectation. Identifying this distinction early helped expose a feasibility question before investing in an unsuitable approach. I therefore see the development plan as a starting point that makes assumptions visible and guides early investigation, while remaining open to revision as we learn.

   Moustafa Moustafa:

   A development plan makes it clear who is responsible for what before the work starts. Without one, tasks are unclear and it is easy for someone to say "I don't want to do this" with nothing to point back to. When we

define roles, who works on what, how we split tasks, and which tools each person uses, it saves a lot of friction later.

I have learned this from experience. On a previous project, we did not have a well-defined plan, and it came back to haunt us near the end. All three of us got busy with university just as some milestones were due. Nobody wanted to take them on, but someone had to, and the work ended up being split unevenly accross 2 people. With a clear plan, clearly separated tasks, and agreed roles, we could have avoided that situation entirely. That is why I think having the plan in place from the start matters so much for a five-person capstone team.

Rayan Nasrallah:

The benefit I found clearest was being made to sketch the technical implementation at a high level, because doing so exposed what we did not yet know how to build. Committing to a rough shape for the system forced questions that a feature list never raises on its own. Our discussion about keeping refined drawings editable is a good example: the requirement sounds straightforward until you consider the implementation, at which point returning a generated image clearly fails to satisfy it. Identifying that early gives us time to investigate, rather than discovering the problem once work depends on the wrong approach.

These feasibility questions feed directly into scope, which matters more here than it would on an open-ended project. We have two semesters, five members, and a fixed set of deliverables alongside the implementation, so the amount of work a feature requires is as important as how valuable it is. Thinking this through in advance is what let us treat structured practice, adaptive guidance, and progress reporting as core work while moving camera-based tracing and letter activities to stretch goals. Without estimating the effort first, that judgement would have been guesswork.

I also think there is a real difference between discussing these decisions and writing them down. Discussion can produce the feeling of agreement while members leave with different understandings of what was decided, and writing forces the vagueness to surface. The written plan also gives us a concrete reference to revise. I do expect parts of it to prove wrong once implementation begins, since we are planning around technology we have not used yet, so I see its value less in being accurate and more in making our assumptions explicit enough to correct.

Muhammad Huzaifah:

I see the development plan as a way to establish a shared high-level vision before moving into detailed implementation. Agreeing early on responsibilities, workflows, testing, and scheduling helps us understand how our individual contributions fit together and avoids conflicting assumptions. Recording these decisions and their reasons also gives us traceability: when we revise an approach, we can explain what changed and why. For me, the value is in making informed decisions early while keeping them open to

revision as we learn. This gives the team a common starting point without treating the initial plan as fixed.

Our discussions about commit and pull request conventions made this practical for me. We shared the same project goals but had different expectations about how contributions should be recorded and reviewed. Agreeing on these conventions showed me why working expectations need to be explicit early, so that differences do not repeatedly interrupt implementation later.

Shamil Canbolat:

Every team members goes into a project with different underlying assumptions. If these are not expressed, discussed, compromised, and agreed upon by the whole group before the project starts, problems in development will occur down the line. For example, meeting schedules ensures that team members can frequently check in for accountability and clearing up confusion while defining roles makes it clear what exactly each person is accountable for.

It was very beneficial for our project that we outlined communication rules, workflow, and expected technology. Github has many features and we needed a standard of using them for consistency and to minimize confusion. Our workflow for creating issues, generating PDFs, and reviewing merge requests makes things run very smoothly. In addition, defining technology is important because in a group setting, people have diverse backgrounds and may not all be familiar with the same tools, framworks, or libraries.

Overall, deciding on the structure of a project beforehand makes sure that incorrect development methods are addressed early on. Such problems like bad coding standards or suboptimal documentation could affect traceability efficiency.

2. **In your opinion, what are the advantages and disadvantages of using CI/CD?**

   Ahmed Elzaria:

   I see the main advantage of Continuous Integration (CI) as giving the team consistent feedback whenever changes are proposed. With five members contributing, relying on each person to remember and reproduce every check would make mistakes easier to miss. Automated builds, linting, and tests can identify problems while the relevant change is still small and easier to investigate. For our documentation, an automated LaTeX build can also reveal compilation errors before submission.

   Continuous Delivery or Deployment (CD) extends this process. Continuous delivery keeps changes ready for release, while continuous deployment automatically releases changes that pass the required checks. For Squigglr, preview deployments would make it easier for teammates to review an interface change and try it on a tablet. However, passing automated checks should not be treated as proof that a change is ready for users. Drawing

responsiveness and stylus behaviour still need evaluation on actual devices, and a successful document build does not establish that its content satisfies the rubric.

The main disadvantages are the effort required to configure and maintain the workflows, the time spent investigating failed runs, and the risk of trusting incomplete tests too much. A pipeline can pass while important behaviour remains untested. For example, a scoring function could produce consistently incorrect results if its tests use the same mistaken assumptions as its implementation.

Given our timeline, I favour introducing a small set of useful checks first and expanding them as the application develops. This would allow us to gain early feedback without spending disproportionate effort on automation before the core functionality exists. Human review and device testing would remain part of our process.

Moustafa Moustafa:

I first saw the real value of CI/CD during my previous co-op. It was my first experience on a large project, and I realized that a codebase of that size could not function without it once more than a few people were contributing. The same applies on a smaller scale to capstone. With five people working in the same repository, we would constantly run into conflicts and broken builds without CI/CD, and it would be hard to tell which change caused a problem. Automated checks on every pull request catch issues early, make it clear which change introduced them, and make it much easier to roll back when something goes wrong.

The main disadvantage is that it can become a bit of a rabbit hole. Some people keep improving their pipeline, adding more optimizations and tools, until maintaining the CI/CD takes more time than the actual work. There is a balance to find: the pipeline should be good enough to let the team work together safely and recover from mistakes, without being over-engineered. For us, that means keeping CI/CD simple and only adding to it when we actually need to.

Rayan Nasrallah:

Continuous Integration means merging our work into the shared branch frequently, with automated checks running on it each time, instead of integrating large amounts of work at once. Continuous Delivery keeps every change that passes those checks ready to release, while continuous deployment releases it without waiting for a person. Our repository already does all of this for documentation: a GitHub Actions workflow determines which documents a change affects, compiles them, commits the resulting PDFs, and publishes them to our project page. Once implementation begins, the same pipeline would run tests and linting rather than only document builds.

The advantage I value most is the effect this has on how we work, rather than what it automates. Because every change arrives as a pull request

that has to build before it can merge, work cannot sit unintegrated on a branch for weeks and then arrive all at once near a deadline. Changes stay small enough for somebody other than the author to read properly, and the project advances in increments we can each follow.

The published output stays current for the same reason. Our PDFs always correspond to what is on the main branch, so nobody has to compile everything before a submission and discover a problem then. For the application, preview deployments would let the rest of the team try an interface change on a tablet instead of taking the author's description of it on trust.

I initially thought there were no real disadvantages, and looking more carefully at our own workflow changed that view. The clearest cost is that a passing pipeline is easy to mistake for a correct result. Our problem statement compiled without error while citing references the document did not yet contain, because LaTeX has no reason to object to that. The same failure will be available to us in code, where tests can pass while drawing responsiveness on a real tablet remains wrong. Automated checks only cover what we thought to check, so I read a passing run as evidence that nothing obvious broke rather than that the work is right.

Muhammad Huzaifah:

I view CI/CD as a technical investment. Setting up reliable builds, tests, and deployment workflows takes valuable time upfront, and those workflows need ongoing maintenance. However, that effort can accelerate later development by catching bugs early, reducing repetitive manual work, and making releases more consistent. Automated rollback, when configured and tested, can also reduce the impact of a failed deployment. The trade-off is that poor configuration or incomplete tests can give false confidence. I would therefore start with checks that address our most immediate risks and expand them as the project grows, while retaining human review.

Our documentation pipeline made this risk concrete: it reported success while uploading an outdated PDF because change detection failed and compilation was skipped. This showed me that a successful status is only useful if the intended checks actually ran. For future workflows, I would verify both failure handling and the generated output, rather than relying on the status alone.

Shamil Canbolat:

CI/CD did not seem worth it to me at the beginning because the disadvantages are more apparent at first than the disadvantages. CI/CD can seem like a waste of time, requiring effort to configure and time to get used to using. It can be bloated and slow, requiring builds and automated testing for simple commits. As the development continues, time must be allocated to update and maintain checks and deployment tests.

But the advantages outweigh the disadvantages. Bugs are caught early, prevented more time being wasted on debugging later on. In addition to

saving time by eliminating manual builds and limiting human validation, deployment tests force you to begin thinking of edge cases, risks, and output domain early on, making later system testing easier. Builds, in our case PDF generation, has been very beneficial for catching formatting errors, seeing blame with highlighted colors, and comparing revisions to older commits. False positives and negatives are bound to occur, but this will allow us to improve the deployment and improve the robustness of the checking each time we catch a mistake with the pipeline.

3. **What disagreements did your group have in this deliverable, if any, and how did you resolve them?**

   Team response:

   Our main disagreements concerned team member roles, expectations, and the consequences of missing meetings or commitments. We wanted clear accountability, but we also recognised that overly rigid rules could treat unavoidable circumstances as a lack of commitment. Through discussion, we distinguished between situations communicated in advance, emergencies, and repeated unexplained absences or missed deadlines.

   We resolved this by establishing clear expectations for attendance, communication, and task completion, alongside a staged response to recurring problems. When an accountability trigger is reached, the first step is a private discussion to understand the cause, followed by a recorded recovery plan with specific tasks, support, and deadlines. We review the plan within one week and seek guidance from the teaching team if commitments remain unmet without an acceptable explanation or the problem recurs during the following milestone. This approach makes accountability more predictable while allowing us to consider individual circumstances.

   We also differed on how specific our roles should be. Narrow assignments risked limiting members to particular tasks, while broad shared roles made it unclear who should coordinate decisions or follow up on gaps. We settled on named leads for key areas while retaining software development, testing, documentation, and peer review as shared responsibilities. Leads coordinate their areas without becoming solely responsible for all related work. Assigning Moustafa as the notetaker provides a consistent owner for meeting records, while rotating the milestone quality assurance coordinator distributes final review duties.

   These discussions showed us that fairness depends on the effort and responsibility involved, rather than simply giving everyone the same number of titles. They also revealed that shared intentions do not necessarily imply shared expectations. In future milestones, we will review the workload and clarify ownership earlier, using our experience to adjust responsibilities before uncertainty becomes a source of conflict.

Appendix — Team Charter

This charter sets out how the Squigglr team will work together, distribute responsibilities, and address problems. All five members will review and agree to these commitments before submission. Changes will be discussed by the team and recorded in meeting notes.

External Goals

The team has three goals:

- Complete a project that we are proud of and can confidently present as a team. We want our work to meaningfully address the problem and goals identified in the Problem Statement and Goals document, with each member taking ownership of the decisions and contributions that shape the final result.
- Extend our learning by taking a software project from initial planning and requirements through design, implementation, testing, and final demonstration. Each member will have opportunities to apply existing skills, explore unfamiliar areas, and gain experience collaborating throughout the development process.
- Earn a high grade that reflects the quality of our work, our individual contributions, and our growth as a team.

Attendance

Reliable attendance supports shared understanding and timely decisions. The following expectations apply to scheduled team meetings, the weekly meeting with the project supervisor, Dr. Anand, and meetings with the teaching team. All five members are expected to attend these meetings, subject to the absence and emergency provisions below.

Expectations

Members are expected to:

- attend all agreed meetings;
- arrive on time;
- remain for the scheduled duration;
- communicate known absences, late arrivals, or early departures at least 24 hours in advance whenever possible.

Meeting times will be agreed with consideration for members' availability.

Moustafa, or the assigned substitute notetaker, will record attendance and action items. A member who misses a meeting will review the notes and confirm their assigned actions within 24 hours of the notes being available, unless an emergency prevents this.

Acceptable Excuse

Acceptable reasons for missing a meeting or renegotiating a deadline include:

- illness;
- family emergencies;
- religious observances;
- essential appointments;
- unavoidable academic conflicts.

Predictable conflicts should be raised early so that the team can adjust its schedule or redistribute work.

The following will not normally excuse an absence or missed deadline:

- forgetting a meeting;
- leaving work until the last moment;
- accepting a conflicting optional commitment without notifying the team.

Members are expected to plan around known coursework and request help when their workload becomes unmanageable.

In Case of Emergency

1. A member experiencing an emergency will notify the team as soon as reasonably possible. They need only communicate their availability and the work affected; private personal details are not required.
2. The Project Manager will coordinate temporary reassignment of urgent tasks. Available work and relevant notes should be shared when the member is able to do so.
3. If the disruption threatens a course deadline or substantially affects the team's capacity, the Team Liaison will contact the teaching team for guidance.

Recovery expectations will account for the member's circumstances.

Accountability and Teamwork

All members share responsibility for a respectful working environment and reliable contributions. The lead roles in Section 6 support coordination, while each member remains accountable for their assigned work.

Quality

Members will arrive at meetings having reviewed the agenda and relevant material, with a clear update on:

- progress;
- blockers;
- decisions needed.

Submitted work should satisfy the completion criteria in its GitHub issue and be checked by its author before review.

The team will aim to have review-ready contributions completed at least 48 hours before each course deadline, leaving time for integration and corrections. Different internal deadlines may be agreed when dependencies require them, but they must be recorded before the existing deadline passes.

Reviews follow these rules:

- Code changes will include appropriate tests, and documentation will be checked against the applicable writing checklist and rubric.
- Each substantial contribution will receive review from another member.
- The milestone Quality Assurance (QA) coordinator will rotate each milestone and track unresolved review findings.
- Authors remain responsible for addressing feedback on their work.

Attitude

In discussions, members will:

- listen without interrupting;
- give constructive feedback and challenge ideas respectfully;
- treat questions and requests for help as opportunities to resolve problems early;
- acknowledge others' contributions;
- report mistakes or uncertainties honestly.

Disagreements are handled in these steps:

1. The affected members discuss the disagreement first, focusing on the specific issue and its effect on the project.
2. If it is unresolved, the Project Manager will facilitate a discussion; another agreed member will facilitate if the Project Manager is involved.
3. Persistent interpersonal problems will be referred to the teaching team.

Project decisions and actions will be recorded without placing sensitive personal details in the public repository.

Stay on Track

The team will plan around an average contribution of approximately ten hours per member per week, including scheduled course activities. Weekly task allocation will account for availability and the effort required for implementation, research, coordination, and review.

Every substantive task will have an owner, completion criteria, and a deadline in GitHub. Members will update their assigned issues before each team meeting and report blockers as soon as they become apparent. Progress will be reviewed weekly using completed work, review participation, attendance, and outstanding tasks. Commit counts alone will not determine whether contributions are fair.

The following events will trigger a workload and accountability discussion:

- Two unexcused meeting absences within four weeks.
- Three arrivals more than ten minutes late within four weeks, without prior notice or an acceptable reason.
- Two missed internal deadlines within four weeks, without an agreed extension or an acceptable reason.
- No substantive progress on assigned work across two consecutive weekly reviews, without a communicated blocker or agreed adjustment.

After a trigger, the team takes these steps:

1. A private discussion to understand the cause.
2. A recorded recovery plan with specific tasks, support, and deadlines.
3. A review of that plan within one week.
4. If commitments remain unmet without an acceptable explanation, or the problem recurs during the following milestone, the Team Liaison will seek guidance from the teaching team.

Urgent work may be reassigned to protect the project, with contributions recorded accurately.

Members who complete work early or provide valuable support will be acknowledged during meetings. Spare capacity will be used for peer assistance, learning, or reducing later workload, rather than automatically assigning more work to the same members.

Team Building

Meetings will include a brief personal check-in to help members communicate workload concerns and maintain positive relationships. The team will aim to hold an optional social activity or informal celebration after each major milestone. Activities will be inexpensive and accommodate members' preferences and availability.

Occasional paired work sessions will provide opportunities to learn from one another and build confidence in unfamiliar parts of the project.

Decision Making

Significant decisions about scope, technology, scheduling, and working practices are made in these steps:

1. The team first seeks consensus. Members will have an opportunity to explain their views, supported by project requirements, evidence, and practical constraints.

2. If consensus cannot be reached in time, a decision requires support from at least three of the five members. Absent members will be given an opportunity to provide input asynchronously.

3. Unresolved decisions that threaten a milestone will be raised with the project supervisor or teaching team.

Routine implementation choices may be made by the assigned developer in consultation with the relevant lead, provided they remain within agreed requirements and interfaces. Decisions affecting other members' work will be communicated before dependent work proceeds. Significant decisions and their rationale will be recorded in GitHub issues or meeting notes.